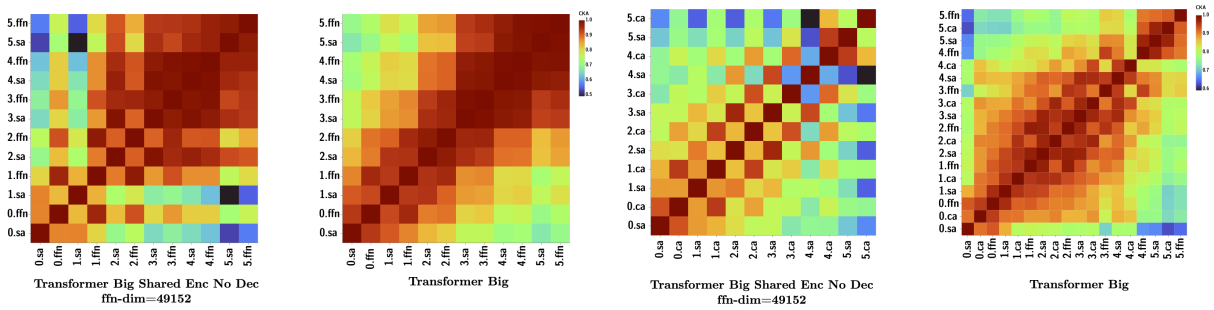

(a) Encoder self similarity.

(b) Decoder self similarity.

Figure 1: CKA self similarity of encoder and decoder layers of the *One Wide Encoder* model vs. the Transformer Big baseline. We identify each component with a label: `index.name`. For example, `0.sa` refers to the self-attention on layer 0, while `4.ca` refers to the cross-attention on layer 4.

ing runs of our baseline, individual runs can have a normalized score above 100%.

Table 5 shows normalized similarity scores for several models. Under the Encoder columns we compare the encoder representations, and under the Decoder columns we compare decoder representations. Sharing FFNs leads to consistenly lower (normalized) similarity scores than models that do not share, both in terms of internal representation (CKA) and semantic spaces (LNS). As shown, although models that share FFNs have lower similarity scores compared to those that do not, the scores are still very close to 100%. Moreover, these decreases align with the drops in BLEU seen in Table 2, where the model with the lowest similarity score (`ShareEncDec`) is also the least accurate model. We observe a similar trend for models that drop the FFNs in the encoder or decoder, these models exhibit lower similarity scores with the respective component than models sharing them, as shown by `NoEnc` and `NoDec`. In addition, the former result again suggests the FFNs in the encoder are more important than in the decoder as the similarity shifts drastically compared to all other settings.

For completeness, we report on the last row the similarity scores for the One Wide FFN model, which is more accurate than the base model. The internal representations generated by that model diverge from those of the base model. Interestingly, we observe a larger drop in LNS scores than in CKA scores, indicating that the shift occurs mostly in semantic space, rather than the Euclidean space captured by CKA. For a detailed layer-wise similarity analysis that breaks out the aggregate analysis in Table 5 see Appendix C.2.

4.4.2 A Qualitative View of Redundancy

We now study into the impact of our One Wide FFN model on the redundancy of the internal representations. In addition to adopting their definition of redundancy, we also adopt Dalvi et al. (2020)'s method of computing self-similarity, namely looking at how the representations change as they go through each module (self-attention, FFN, or cross-attention) of the model. In particular, we use CKA to compute similarity between the output of different modules within the same model.

In Figure 1a, we show the CKA self-similarity matrices for the encoders of the One Wide FFN model and the `Transformer Big`. We do the same for the decoders in Figure 1b. These matrices show how similar each module of the network is to all other modules *within that network*. The diagonal of the matrix is the similarity between a module and itself and is always 1.

As shown, there is high similarity between adjacent modules of the `Transformer Big`, both on the encoder and decoder, indicated by areas with darker red around the diagonal. The prevalence of high similarity patterns among adjacent modules suggests a substantial degree of redundancy, and eliminating a module has a negligible impact on the final representations. On the other hand, we observe a distinct checkerboard pattern on the self-similarity matrices of the One Wide FFN model, where individual modules tend to exhibit lower similarity with their immediate neighbors than with their second neighbors (i.e., the neighbors of the neighbors). On the encoder, the checkerboard pattern emerges especially in the earlier modules while on the decoder, that pattern appears more consistently throughout the layers. This pattern gives an indication that our model is learning non-trivial

transformations of the input, leading to decreased redundancy within the network.

4.5 Other architectures and Languages

So far, all our experiments focused on the Transformer Big and on WMT22 EN →DE. In this section, we apply what we learned to other architectures and language pairs. We run experiments on the low resource language direction EN →RO and a large scale multilingual model.

For EN →DE, we apply our proposal to a Transformer Base model, a Deep Encoder Shallow Decoder model (Kasai et al., 2021), and a Decoder-Only model. For the Transformer Base, we observe an accuracy gain of 0.5 BLEU (2.2 BLEU over the vanilla SharedEncNoDec model) and an inference speedup of around 25%. In the Deep Encoder Shallow Decoder model, we observe a more modest accuracy gain of 0.2 BLEU points (0.9 BLEU over the vanilla SharedEncNoDec model). However, the inference speedup from dropping the decoder FFNs is minimal ($< 1\%$), which is expected because of the small depth of the decoder in this architecture.

Decoder-only models With the advent of Large Language Models (LLMs) like GPT (Brown et al., 2020), and PaLM (Chowdhery et al., 2022), a lot of effort has been put on decoder-only Transformer models. We train a decoder-only model on WMT22 EN →DE, as shown on Table 6. Due to the absence of an encoder, we are limited to applying a wide FFN on the decoder side. As in the other setups, we get an accuracy gain of +0.3 BLEU over the baseline decoder-only model (+1.7 BLEU over ShareDec), but the latency degrades by 12%. This is not surprising: due to the autoregressive nature of the decoder, increasing the size of its FFN has a bigger impact on speed.

Low-resource languages In EN →RO the accuracy of the One Wide FFN Model is only on par compared to the base model, even though it is a higher than the vanilla SharedEncNoDec model. We hypothesize that due to the low resource condition, our proposed model already reaches saturation as there are not that many salient textual patterns to be learned by the FFN.

Multilingual Finally, we observe the similar trend on the multilingual setup, where the One Wide FFN Model is +1.2 SPBLEU points more accurate than the baseline Transformer Big and +2.5 SPBLEU points more accurate than the vanilla SharedEncNoDec, this gain is significant in 79 out of 90 directions and when all tests sets are concatenated. Additionally, this large accuracy gain also comes with around 18% inference speed-up, consistent with our previous results.

5 Related Work

Weight pruning and parameter sharing are well-known techniques to reduce a model's footprint. Given the scale of the latest models (Chowdhery et al., 2022), there have been multiple efforts to prune neurons based on different automatic methods (Dalvi et al., 2020; Michel et al., 2019; Voita et al., 2019), sharing parameters efficiently (Ge et al., 2022; Reid et al., 2021), and factorizing certain components (Lan et al., 2020; Hu et al., 2022).

Neuron pruning methods often focus on finding and pruning redundant neurons through correlation methods (Dalvi et al., 2020), but also on how Transformer components like the multi-head attention can be pruned significantly due to model redundancy in the encoder or decoder either by checking the gradients salience (Michel et al., 2019) or a differentiable relaxation of the l_0 regularization at training time (Voita et al., 2019).

For parameter sharing, the Universal Transformer (Dehghani et al., 2019) proposed a model where all layers are shared (i.e., in effect it reduced the model to a single shared layer). Takase and Kiyono (2023) proposes finding an optimal configuration of shared layers in the encoder or decoder through different methods of sharing (in sequence, in cycle, or in reversed cycle) always keeping a specified number of final layers[9]. Similarly, Reid et al. (2021) proposes an approach where just the middle layers are shared, while the bottom and top layers are independent, and using a lower dimensionality for the embedding layer. Analogously, Ge et al. (2022) focus on minimizing the number of parameters and the number of calls to each parameters' group in order to optimise on-device models. They achieve this by sharing the encoder and decoder in a similar way to both previous methods, particularly by sharing all layer parameters in cycle like Takase and Kiyono (2023).

Previous works also focus on reducing the dimensionality of certain parameters, mostly through low rank factorization. Lan et al. (2020) decomposes the embedding layer into a lower rank em-

[9] See Appendix A for a detailed description and comparison.

	BLEU	CHRF	COMET	Speed	$\lvert\theta\rvert$ (%)
Transformer Base EN →DE	34.2	61.6	54.1	$116.3^{\pm 0.9}$	70M (100)
+ SharedEncNoDec FFN $d_{ff'} = 2,048$	$32.5^{\dagger}$	$60.1^{\dagger}$	50.0	$146.0^{\pm 1.6}$	47M (67)
+ SharedEncNoDec FFN $d_{ff'} = 24,576$	**34.7**	**61.8**	**55.6**	$146.8^{\pm 1.3}$	70M (100)
Transformer Decoder-Only EN →DE	35.8	62.8	57.7	$79.8^{\pm 1.9}$	202M (100)
+ ShareDec FFN $d_{ff'} = 4,096$	$34.4^{\dagger}$	$61.7^{\dagger}$	54.1	$79.7^{\pm 1.3}$	110M (48)
+ ShareDec FFN $d_{ff'} = 49,152$	**36.1**	**62.9**	**59.4**	$69.3^{\pm 0.2}$	202M (100)
Transformer Deep Enc. Shallow Dec. EN →DE	35.5	**62.4**	58.0	$230.1^{\pm 0.8}$	236M (100)
+ ShareEncNoDec FFN $d_{ff'} = 4,096$	$34.8^{\dagger}$	$61.6^{\dagger}$	55.4	$235.0^{\pm 0.5}$	127M (54)
+ ShareEncNoDec FFN $d_{ff'} = 57,344$	**35.7**	**62.4**	**58.9**	$233.5^{\pm 0.7}$	236M (100)
Transformer Base EN →RO	**22.9**	**52.9**	**50.9**	$119.3^{\pm 1.1}$	64M (100)
+ SharedEncNoDec FFN $d_{ff'} = 2,048$	$22.2^{\dagger}$	$52.5^{\dagger}$	45.8	$152.8^{\pm 1.4}$	41M (64)
+ SharedEncNoDec FFN $d_{ff'} = 24,576$	**22.9**	52.8	46.7	$150.6^{\pm 0.5}$	64M (100)
Transformer Big Multilingual	26.8	46.3	47.7	$94.6^{\pm 1.6}$	422M (100)
+ SharedEncNoDec FFN $d_{ff'} = 4,096$	$25.5^{\dagger}$	$45.1^{\dagger}$	40.8	$107.1^{\pm 1.4}$	330M (78)
+ SharedEncNoDec FFN $d_{ff'} = 49,152$	$\mathbf{28.0}^{\dagger}$	$\mathbf{47.3}^{\dagger}$	**50.7**	$111.5^{\pm 1.1}$	422M (100)

Table 6: Accuracy of One Wide FFN for EN →DE with Transformer Base, Decoder Only, and Deep Encoder Shallow Decoder on WMT22; for low resource EN →RO with Base version on WMT16, and multilingual with Transformer big on Flores. † implies the system is statistical significantly different at $p < 0.05$.

bedding matrix and a projection to the actual hidden size while also sharing all parameters across all layers. In addition to sharing parameters efficiently, Ge et al. (2022) proposes a lightweight decomposition of the FFN where instead of a single component there are 2 projections with a smaller dimensionality than vanilla Transformers. Our work is close to Ge et al. (2022) but instead of factorizing we explore sharing and full pruning of the FFN. In contrast with previous works, we also explore increasing the encoder FFN size while dropping the decoder's completely.

6 Conclusion

In this work, we studied the importance of the FFN in Transformer models. We analyzed the impact of removing and/or sharing the FFN across layers and found that, due to this component's redundancy, the model sizes can be substantially reduced with little impact on accuracy for Machine Translation. In particular, we found that sharing the FFN across all encoder layers while making it larger and removing it from the decoder layers leads to models that are more accurate and faster at inference.

Our findings are applicable across multiple settings, including decoder-only and multilingual models. In a low-resource setting the results are modest but our approach can still recover the baseline's performance with a faster inference.

Finally, we conducted a thorough similarity analysis between the vanilla Transformer and our proposed architectures, and found that the latter's internal representations do not differ significantly from the former's, except in that they are less redundant.

Limitations

In this work, our focus was Machine Translation. Although we expect the results to generalize to other sequence-to-sequence tasks, further experiments are needed, which we leave for future work.

Ethics Statement

One important consideration is the energy consumption for model training, which results in greenhouse emissions (Strubell et al., 2019). Our work uses existing datasets, and inherits some of the risks associated with them, such as privacy leakage (Carlini et al., 2021) and gender bias (Cho et al., 2019). Mitigation strategies such as those from Vanmassenhove et al. (2018) may be necessary.

Acknowledgements

We would like to thank Robin Schmidt, Matthias Sperber, and Stephan Peitz for their feedback and support in reviewing this work.